Token Retrieval from HTTP-Only Cookies (Updated Architecture)

Goal

Move authentication from:

- frontend-managed JWT tokens
- Zustand/localStorage token persistence

to:

- secure HTTP-only cookies
- backend-controlled session lifecycle
- multi-subdomain authentication support

while supporting:

- root domain super admin login
- tenant subdomain login
- automatic refresh token rotation
- local multi-tenant development using `lvh.me`

Final Architecture

Authentication Ownership

OLD ARCHITECTURE

- Frontend owns authentication state

Frontend stored:

- access token
- refresh token

Frontend manually:

- attached Authorization headers
- refreshed tokens
- persisted sessions

NEW ARCHITECTURE

- Backend owns authentication state

Browser automatically sends:

- access_token cookie
- refresh_token cookie

Frontend only stores:

- cached user data
- UI auth state

Frontend no longer stores tokens anywhere.

Login Flow

Root Domain Login

User visits:

- http://lvh.me:3000

User enters:

- superadmin@azisly.local

Backend identifies:

- is_super_admin = true

User remains on:

- lvh.me

College User Login

User visits:

- http://lvh.me:3000

User enters:

- college.admin@demo.local

Backend:

1. extracts email domain
2. resolves matching college
3. determines tenant slug
4. returns tenant slug in response

Frontend redirects to:

- http://demo.lvh.me:3000

Important Architectural Change

Slug Resolution Moved Fully to Backend

OLD FLOW

Frontend guessed:

- tenant slug
- email domain mapping

This was fragile and duplicated business logic.

NEW FLOW

Backend fully resolves tenant.

Frontend only sends:

- {
- "email": "...",
- "password": "...",
- "is_super_admin": false
- }

Backend:

- determines college
- validates tenant
- returns resolved slug

This is the correct architecture.

Backend Login Controller Logic

Super Admin Resolution

Backend checks:

- public.super_admins

using:

- SELECT * FROM super_admins
- WHERE email = $1
- LIMIT 1

College Resolution

Backend:

1. extracts email domain
2. generates domain candidates

Example:

- college.admin.cs.demo.local

becomes:

- [
- "college.admin.cs.demo.local",
- "cs.demo.local",
- "demo.local"
-]

Then:

- SELECT *
- FROM colleges
- WHERE domain IN (...)

Longest matching domain wins.

User Lookup

Once college is resolved:

- SELECT *
- FROM "tenant_demo".users
- WHERE email = $1
- LIMIT 1

Dynamic schema:

- collegeRecord.schema_name

Cookie Architecture

Cookies Set By Backend

Access Token Cookie

- res.cookie("access_token", access_token, {
- httpOnly: true,
- secure: false,
- sameSite: "lax",
- domain: ".lvh.me",
- path: "/",
- maxAge: 1000 * 60 * 60,
- })

Refresh Token Cookie

- res.cookie("refresh_token", refresh_token, {

- httpOnly: true,
- secure: false,
- sameSite: "lax",
- domain: ".lvh.me",
- path: "/",
- maxAge: 1000 * 60 * 60 * 24 * 7,
- })

Why `.lvh.me` Domain Matters

`lvh.me` automatically resolves to:

- 127.0.0.1

Supports:

- demo.lvh.me
- abc.lvh.me
- xyz.lvh.me

This allows:

- real subdomain testing locally
- shared cookies across subdomains

Without hacks.

Frontend Authentication Changes

Tokens Removed From Zustand

OLD

- accessToken
- refreshToken
- setTokens()

NEW

Removed completely.

Frontend should NOT store tokens anymore.

AuthGuard Architecture

OLD

AuthGuard:

- managed refresh logic
- stored tokens
- restored sessions

NEW

AuthGuard:

- only hydrates current user
- validates role access
- redirects if unauthenticated

Authentication lifecycle now belongs to:

- apiClient()

Centralized API Client

Important Final Decision

There should be:

- ONE auth-aware API client

Not:

- authenticatedFetch
- authenticatedApiClient
- apiClient

all separately owning refresh logic.

Final Responsibility

`apiClient()` handles:

- cookies
- refresh
- retries
- logout
- auth errors

Everything else should build on top of it.

Refresh Token Flow

Trigger

If:

- response.status === 401

then:

1. call `/auth/refresh`
2. backend verifies refresh cookie
3. backend sets fresh cookies
4. original request retries

Refresh Controller

OLD

Frontend sent:

- {
- "refresh_token": "..."
- }

NEW

Backend reads:

- req.cookies.refresh_token

This is critical.

Frontend never sees refresh token now.

Refresh Token Security

Refresh endpoint should validate:

- payload.token type === "refresh"

Otherwise access tokens could be reused as refresh tokens.

Frontend Redirect Logic

OLD

Hardcoded:

- demo.lvh.me

NEW

Derived dynamically:

- const hostname = window.location.hostname
- const rootDomain = hostname
- .split(".")
- .slice(-2)
- .join(".")

Then:

- ${subdomain}.${rootDomain}

This supports:

- local
- staging

- production

without hardcoding.

Seed System

Purpose

Seed system creates:

- local tenants
- users
- departments
- students
- roles
- realistic development environment

Important Seed Fixes

Password Column Mismatch

Problem:

- password_hash does not exist

Cause:
DB schema used:

- encrypted_password

but auth code expected:

- password_hash

Final decision:

- standardize on password_hash

Required Infrastructure Files

Must commit:

- `seed.local.ts`
- `src/config/db.ts`
- `knexfile.js`
- migration files
- SQL extension files

Do NOT commit:

- `.env`

Commit:

- .env.example

instead.

PostgreSQL Local Setup

Required ENV

- PGSQL_DATABASE_URL=postgres://azisly:azisly@localhost:5433/azisly_dev

---

Common Failure

Error

- getaddrinfo ENOTFOUND base

Cause:
Wrong DB config still referencing old container hostname.

Correct DB Setup

- const pool = new Pool({
- _connectionString: process.env.PGSQL_DATABASE_URL,
- })

Required Dev Commands

Reset DB

- npm run db:down

Start DB

- npm run db:up

Seed Local DB

- npm run seed:local

Security Improvements Still Pending

1. Refresh Race Prevention

Currently:
multiple simultaneous 401s
→ multiple refresh calls

Future improvement:

- shared refresh promise / mutex

---

2. Production Cookie Security

Currently:

- secure: false

Production should use:

- secure: true

---

3. Seed Script Safety

Protect against accidental production execution:

- if (!databaseUrl.includes("localhost")) {
- throw new Error(...)
- }

Final Mental Model

OLD

- Frontend authentication system

NEW

- Backend session system with frontend user cache

That distinction changes:

- token handling
- refresh flow
- auth guards
- state management
- API architecture
- security boundaries

And most migration bugs came from the codebase being half in one model and half in the other.